

Test Completion Report

eBay Web Application & Library-DB-Express Testing Project

Course: SE 2226 – Software Testing

Prepared By: Group 5

Team Members:

- Alphan Tekin – 23070006074
 - Uday Beyaz – 23070006061
 - Talha İşler – 23070006080
 - Baran Kımaz – 24070006094
 - Furkan Gencer – 22070006039
-

1. Summary of Testing Performed

This report summarizes the testing activities carried out for the SE 2226 Software Testing term project. The project involved testing two separate systems with different testing approaches and objectives:

- eBay Web Application (System / UI Testing)

- Library-DB-Express Backend Application (Unit & White-box Testing) The eBay web application was tested mainly through black-box testing techniques. Manual testing activities focused on validating critical user flows such as product search, filtering, sign-in behavior, cart interactions, watchlist actions, and Buy Now workflows. Test design techniques including Equivalence Class Partitioning (ECP), Boundary Value Analysis (BVA), and Decision Table Testing (DTT) were systematically applied to ensure structured and meaningful test coverage. Seven major manual test groups were prepared and executed. These included:
 - Search bar input validation
 - Sign-in validation
 - Search result sorting verification
 - Price filter boundary tests
 - Cart quantity boundary tests
 - Sign-in logic combinations
 - Cart / Watchlist / Buy Now logic combinationsAutomation testing was performed using Selenium IDE and Selenium WebDriver. Automated scenarios included login operations, product search workflows, and cart-related flows. Selenium IDE scripts were later converted into WebDriver-based automation code using Java and Maven-based project structures. For the Library-DB-Express application, white-box testing techniques were applied using Jest and Supertest frameworks. The main focus was validating CRUD operations and API responses for Book routes. A total of 14 unit tests were implemented. The project also included coverage analysis and mutation analysis in order to evaluate the effectiveness and reliability of the implemented unit tests.

2. Deviations from Planned Testing

Most planned testing activities were completed successfully according to the original Test Plan. However, several minor deviations occurred during implementation. Advanced non-functional testing activities such as extensive load testing and deep compatibility testing across multiple operating systems were performed at a smaller scale than initially planned. This limitation was mainly caused by time constraints and restricted access to additional testing environments. Additionally, payment confirmation and real checkout workflows on eBay were intentionally excluded from execution in order to avoid real financial transactions and security risks. For the Library-DB-Express application, testing activities focused primarily on Book route operations and their related CRUD functionalities. Some additional backend modules such as extended borrowing workflows and advanced user operations were not tested with the same depth due to limited project duration.

3. Test Completion Evaluation

The testing process substantially satisfied the completion criteria defined in the Test Plan. The following objectives were achieved successfully:

- Manual test cases were designed using ECP, BVA, and Decision Table techniques.
- Selenium IDE tests were executed successfully.
- Automated Selenium IDE tests were converted into WebDriver code.
- Unit testing was implemented using Jest and Supertest.
- Coverage analysis reports were generated successfully.
- Mutation testing was executed successfully using Stryker.
- Presentation and reporting deliverables were completed. No Severity 1 critical failures were observed during executed test scenarios. The performed testing activities validated the core functionality and stability of both tested systems.

4. Factors That Blocked Progress

Several technical and organizational challenges affected the testing process during the project lifecycle. One of the primary issues involved configuring Node.js testing environments and resolving dependency conflicts during Jest and Supertest integration. Early stages of implementation required additional debugging and environment stabilization. Selenium automation testing also experienced synchronization problems because the eBay website contains dynamic page elements and changing UI behaviors. Explicit waits and additional assertion controls were required to stabilize automated execution. During mutation analysis, several mutants caused timeout issues and runtime exceptions. These issues were investigated and reduced through repeated executions and refined test configurations. The project also required coordination between multiple testing activities including manual testing, automation testing, reporting, and presentation preparation. Team collaboration and task distribution played an important role in resolving these challenges.

5. Test Measures

The following measurable testing results were collected throughout the project.

6. Test Deliverables

The following deliverables were produced during the testing project:

- Test Plan Document
 - Manual Test Case Specifications
 - ECP, BVA, and Decision Table documentation
 - Selenium IDE automation scripts (.side files)
 - Selenium WebDriver automation code
 - Unit test source code
 - Coverage analysis reports
 - Mutation analysis reports
 - Jest & Supertest test implementations
 - Presentation slides
 - Final Test Completion Report
- All deliverables were reviewed collectively by the project team before final submission.

7. Lessons Learned

The project provided valuable practical experience in both manual and automated software testing processes. The team gained hands-on experience with formal test design techniques such as ECP, BVA, and Decision Table Testing. These techniques significantly improved the structure and quality of manual test design. Selenium automation activities improved understanding of browser automation, synchronization handling, assertions, waits, and automation architecture. Converting Selenium IDE scripts into WebDriver code also provided practical software engineering experience. The Library-DB-Express testing activities highlighted the importance of unit testing quality rather than simply executing tests successfully. Mutation analysis demonstrated how surviving mutants can expose weaknesses in test logic and identify areas requiring stronger assertions or additional scenarios. The project also improved debugging skills, teamwork, communication, reporting discipline, and understanding of professional software quality assurance practices. Future improvements may include:

- Increasing branch and statement coverage
- Expanding automation scenarios
- Performing advanced performance and security testing
- Extending unit tests to additional backend modules

Coverage Analysis Results

Coverage Type	Result	Interpretation
Lines	88%	Most executable lines were successfully tested.
Branches	73%	Most decision paths were covered.
Statements	65%	Core statements were validated.
Functions	83%	Major backend functions were executed during testing.

Mutation Analysis Results

Metric	Result	Explanation
Total Mutants	81	Total artificial faults generated by Stryker.
Killed Mutants	62	The implemented tests successfully detected.
Survived Mutants	3	A small number of mutants survived.
Timeout Mutants	15	Some mutants caused execution delays.
Runtime Error	1	One mutation produced an unexpected runtime-level failure.
Mutation Score	96.25%	The very high mutation score indicates strong capability.

The mutation analysis results demonstrate that the implemented unit tests were not only executable but also effective at detecting intentionally injected faults. The low number of survived mutants and the high mutation score indicate that the test suite provided strong behavioral validation for the tested backend routes.